

ActuateX 强化学习控制平台——代码修改报告

Learn. Act. Control.

<https://github.com/Functionhx/actuatex>

项目：TinyMal 四足机器人鲁棒运动控制 仓库：<https://github.com/Functionhx/actuatex> 口号：Learn. Act. Control. 报告日期：2026-07-18

1. 报告目的

本报告专门说明本项目相对于开源基线所做的代码修改。重点回答四个问题：改了什么、为什么要改、如何实现、如何验证。完整源码、可直接审阅的补丁、训练与评估脚本及后续更新见

GitHub：<https://github.com/Functionhx/actuatex>。

项目不是简单运行现成示例，而是围绕同一台 TinyMal 四足机器人建立了 Isaac Gym、Isaac Lab（Isaac Sim）和 MuJoCo 三套训练/评估后端，并实现双向 sim2sim、鲁棒性训练、楼梯与推力测试、策略蒸馏和视频录制。大型上游模拟器没有复制进仓库；仓库只保存自主编写的任务、算法、配置、工具及最小兼容补丁，以便清楚区分“上游代码”和“本项目修改”。

2. 修改规模与代码组织

模块	自主代码/补丁	主要作用
Isaac Gym	12 个 TinyMal 环境/配置文件、14 个训练评估脚本、3 个上游补丁	基线行走、推力恢复、楼梯、随机化、混合地形、蒸馏训练
Isaac Lab	12 个任务/算法文件、6 个训练评估脚本、1 个兼容补丁	新版 PhysX 5 原生训练、旧策略迁移、鲁棒与楼梯任务
MuJoCo	14 个训练、模型构建、评估文件	原生 PPO 训练、动力学修正、楼梯/推力测试、反向 sim2sim
公共工具	sim2sim、checkpoint 比较、安装器	统一观测、策略加载、模型筛选、可复现安装

仓库采用“薄集成层”结构：

```
[text]
actuatex/
├── backends/
│   ├── isaac_gym/    # TinyMal 覆盖层与上游补丁
│   ├── isaac_lab/    # Isaac Lab 原生任务、脚本与兼容补丁
│   └── mujoco/       # MuJoCo 原生训练与评估
├── robots/tinymal/  # URDF 与网格模型
├── tools/           # sim2sim 与 checkpoint 工具
├── scripts/         # 可重复执行的上游安装器
└── docs/           # 设计、实验与代码修改说明
```

3. Isaac Gym：从单一行走任务扩展到鲁棒控制任务族

3.1 新增 TinyMal 任务注册

上游 `unitree_rl_gym` 没有 TinyMal。本项目新增

`legged_gym/envs/tinymal/`，并修改任务注册表，使训练入口可以直接选择六种任务：

```
[python]
task_registry.register("tinymal", LeggedRobot, TinyMalCfg(), TinyMalCfgPPO())
task_registry.register("tinymal_push", TinyMalPush, TinyMalPushCfg(), TinyMalPushCfgPPO())
task_registry.register("tinymal_stairs", TinyMalStairs, TinyMalStairsCfg(), TinyMalStairsCfgPPO())
task_registry.register("tinymal_robust", TinyMalRobust, TinyMalRobustCfg(), TinyMalRobustCfgPPO())
task_registry.register("tinymal_robust_stairs", ...)
task_registry.register("tinymal_robust_mixed", ...)
```

对应补丁为：

```
[text]
backends/isaac_gym/patches/0001-register-tinymal-tasks.patch
```

这样做的意义是：不同实验共享同一机器人、观测和控制定义，只改变训练场景与随机化，便于公平对比，而不是复制整个上游仓库后形成不可审阅的“黑箱改版”。

3.2 重新设计机器人控制与奖励配置

`tinymal_config.py` 中完成了以下关键修改：

- 建立 12 自由度默认站姿，按 FL、FR、RL、RR 四条腿分别设置髋、腿、膝关节角度；
- 使用位置控制，`Kp=20`、`Kd=0.5`、`action_scale=0.25`、`decimation=4`；
- 将初始策略标准差从通用四足配置的较大值降低到 `0.3`，避免小型机器人在学习早期因动作扰动过大直接倒地；
- 根据 TinyMal 尺寸重新设置基座高度、接触惩罚、力矩惩罚、关节限位和速度命令范围；
- 将训练并行环境数配置为 4096，以充分利用 GPU 并提高采样吞吐。

控制目标为：

```
[text]
q_target = q_default + action_scale × action
τ = clip(Kp × (q_target - q) - Kd × q̇, -12, 12)
```

这组定义随后被 Isaac Lab 和 MuJoCo 精确复用，是三后端可比的基础。

3.3 新增鲁棒性随机化与专项任务

普通平地策略容易记住单一模拟器参数。为提高迁移能力，本项目新增：

- [tinymal_push.py](#)：在训练或评估期间施加外力，学习扰动后的姿态恢复；
- [tinymal_stairs.py](#)：生成台阶地形并训练连续上台阶；
- [tinymal_robust.py](#)：随机化摩擦、质量、初始姿态、命令和外力；
- [tinymal_robust_stairs.py](#)：将随机动力学与楼梯课程结合；
- [tinymal_robust_mixed.py](#)：混合平地、台阶和扰动，降低对单场景的过拟合。

训练脚本还增加了分阶段训练、checkpoint 热启动、PD 参数消融、持续推力测试、楼梯测试和批量鲁棒性验证。与只看训练 reward 相比，这些脚本给出了跟踪误差、跌倒率、恢复率、姿态角和跨模拟器结果。

3.4 新增无窗口视频录制能力

上游 [BaseTask](#) 在 `headless=True` 时直接把图形设备设为 -1，导致无法在服务器上用相机录制视频。本项目增加 [enable_offscreen_rendering](#)：

```
[python]
self.enable_offscreen_rendering = bool(
    getattr(cfg.env, "enable_offscreen_rendering", False)
)
if self.headless and not self.enable_offscreen_rendering:
    self.graphics_device_id = -1
```

该修改允许“无交互窗口训练”和“离屏相机录制”同时成立，用于生成 PPT 中的平地行走、抗扰和上台阶视频。对应补丁为 [0002-offscreen-rendering.patch](#)。

4. RSL-RL：增加按任务选择的参考策略蒸馏

上游 PPO 只包含策略损失、价值损失和熵正则。本项目增加两个可选的冻结参考 actor，以及独立的观测掩码和权重：

```
[python]
self.reference_actor = None
self.reference_loss_coef = 0.0
```

```
self.reference_mask_fn = None
self.secondary_reference_actor = None
self.secondary_reference_loss_coef = 0.0
self.secondary_reference_mask_fn = None
```

更新时只在指定样本上约束当前策略输出：

```
[python]
reference_loss = mean((mu_current[mask] - action_reference[mask])** 2)
loss = ppo_loss + value_coef * value_loss - entropy_coef * entropy \
      + reference_coef * reference_loss \
      + secondary_coef * secondary_reference_loss
```

该修改解决了多阶段训练中的“灾难性遗忘”：新阶段学习楼梯、随机动力学或大扰动时，参考平地策略可保护已经形成的稳定步态；第二参考策略还可分别保护另一类任务。所有新增功能默认关闭，因此不配置参考 actor 时与上游 PPO 行为一致。完整补丁为：

```
[text]
backends/isaac_gym/patches/0003-reference-policy-distillation.patch
```

5. Isaac Lab / Isaac Sim：不是直接复制，而是重新对齐接口

Isaac Lab 更先进，但“更新”不等于旧策略能直接工作。Isaac Gym 与 Isaac Lab 在物理求解器版本、URDF 导入、驱动器、接触、地面资产、观测接口和默认参数上都有差异。本项目对这些差异逐项处理。

5.1 重建原生 TinyMal Manager-Based 环境

新增 `tinymal_lab/` 包，包含：

- `tinymal_cfg.py`：URDF 导入、关节初值、12 个 IdealPD 执行器和力矩/速度限制；
- `mdp.py`：与旧 Gym 策略兼容的 48 维观测、奖励和事件函数；
- `tinymal_flat_env_cfg.py`：平地迁移与训练；
- `tinymal_robust_env_cfg.py`：随机动力学和抗扰训练；
- `tinymal_stair_env_cfg.py`：原生楼梯环境；
- `tinymal_native_env_cfg.py`：最终原生组合任务；
- `symmetry.py`：左右腿对称变换，用于约束策略结构与数据增强；
- `agents/`：RSL-RL PPO 网络和超参数配置。

5.2 对齐 48 维观测与关节顺序

旧策略的输入为：

```
[text]
```

```
[base_lin_vel(3), base_ang_vel(3), projected_gravity(3), command(3),  
dof_pos_error(12), dof_vel(12), last_action(12)] = 48 维
```

其中缩放、裁剪、四元数方向、关节顺序和上一动作必须完全一致。项目在 `mdp.py` 与公共 `observation_builder.py` 中显式固定这些约定，避免“网络维度相同但语义错位”的隐蔽问题。

5.3 将隐式驱动改为与 Gym 一致的显式力矩规律

初次迁移使用 Isaac Lab 默认隐式关节驱动时，策略能够站住但不产生正确步态。原因是 PhysX 5 的隐式驱动与旧 Gym 每个 physics step 计算并裁剪 PD 力矩并不等价。因此改用

`IdealPDActuatorCfg`，明确设置：

```
[python]  
effort_limit = 12.0  
velocity_limit = 20.0  
stiffness = 20.0  
damping = 0.5
```

这不是简单调一个 reward，而是把底层执行器语义重新对齐。

5.4 修复 Isaac Sim 5.1 URDF 导入和离线地面

测试所用 Isaac Sim 5.1.0-rc.19 缺少某些新版 URDF importer API，并且无 Nucleus 服务时默认网格地面 USD 无法加载。兼容补丁做了两项修改：

- 1 调用 `set_merge_fixed_ignore_inertia` 前先用 `hasattr` 检查，兼容不同 importer 小版本；
- 2 默认地面资产不可达时生成 $2\text{ km} \times 2\text{ km}$ 的薄 `Cuboid`，并正确绑定碰撞材质，使训练可完全离线运行。

完整差异保存在：

```
[text]  
backends/isaac_lab/patches/0001-isaac-sim-5.1-offline-compat.patch
```

6. MuJoCo：从“只能导入 URDF”改成可训练的原生后端

6.1 发现并修复原始 URDF 动力学问题

MuJoCo 直接载入 TinyMal URDF 后存在三个关键问题：无 actuator、关节 armature 与 damping 过小、显式 Euler 下高增益 PD 数值不稳定。表现为机器人即使输入零动作也会从站姿快速塌陷。

`model_builder.py` 对编译后的 MJCF 做了以下修改：

- 为浮动基座加入 `freejoint`；
- 为 12 个关节加入 position actuator；

- 加入 `damping=0.5` 和小转子惯量 `armature=0.01`；
- 使用 `implicitfast` 积分器；
- 设置与 Gym 一致的 `kp=20` 和 `forcerange=[-12, 12]`；
- 支持注入台阶等静态 MJCF 几何体；
- 取消硬编码输出路径，生成模型可按参数保存到 `artifacts/`。

核心生成逻辑如下：

```
[python]
for joint_name in JOINT_NAMES:
    actuator_lines.append(
        f'<position joint="{joint_name}" kp="{kp}" '
        f'forcerange="-{forcelimit} {forcelimit}" />'
    )
```

6.2 新增向量环境和原生 PPO 训练

`mujoco_vec_env.py` 实现多个 MuJoCo 模型并行 rollout，并输出与 Isaac Gym 对齐的观测、奖励、reset 和 extras 接口。`train_mujoco.py` 复用 RSL-RL 的 ActorCritic/PPO，但针对 CPU MuJoCo 批量梯度特点将学习率改为固定 `3e-4`。

这项修改来自实际故障定位：自适应 KL 调度在较小的 1024 环境批量下频繁把学习率压到 `1e-5`，策略长期无法学习。固定学习率避免了该退化，同时保留网络结构和主要 PPO 超参数，使比较仍然公平。

6.3 新增 MuJoCo 专项验收与反向 sim2sim

新增脚本包括：

- `eval_mujoco.py`：六段速度命令跟踪；
- `eval_mujoco_tasks.py`：楼梯和持续外力矩阵；
- `record_mujoco_stairs.py`：录制上台阶视频；
- `evaluate_mujoco_policy_in_isaac.py`：把 MuJoCo 训练策略加载回 Isaac Gym，形成反向 sim2sim；
- `bench_env.py`：向量环境吞吐基准；
- `inspect_model.py`、`test_stand.py`：模型结构和零动作稳定性诊断。

7. 双向 sim2sim 与统一评估

项目同时验证两个方向：

```
[text]
Isaac Gym / Isaac Lab 策略 ——→ MuJoCo
```

为保证公平，三个后端统一了：

- 48 维观测定义与缩放；
- 12 自由度顺序；
- 默认关节角和 action scale；
- 六段命令序列：站立、前进 0.3、前进 0.6、后退、横移、偏航；
- 跌倒判断、速度 RMSE、姿态、力矩和动作统计。

[tools/sim2sim/](#) 负责 checkpoint 读取、NumPy actor 推理、MuJoCo rollout 和三后端比较；[tools/checkpoints/](#) 用硬性门槛筛选模型，而不是只选训练 reward 最高的 checkpoint。

8. 代表性实验验证

以下数字来自本次最终实验记录；它们用于验证修改是否有效，而不是宣称已经完成真实机器人部署。

验证项目	结果	说明
Isaac Gym → MuJoCo 六段平地	平均 RMSE 0.08024, 0/6 跌倒	跨引擎表现最稳定
Isaac Lab → MuJoCo 六段平地	平均 RMSE 0.33272, 2/6 跌倒	暴露 PhysX 5 迁移差异，仍需继续优化
MuJoCo 原生策略平地	平均 RMSE 0.07957, 0/6 跌倒	原生训练已获得稳定步态
MuJoCo 动力学网格	16/16 零跌倒	对质量、摩擦等扰动进行组合验证
MuJoCo 持续推力	11/16 通过	较强或不利方向外力仍是后续重点
Isaac Lab 鲁棒随机测试	平均指标 0.0945, 零 reset	证明原生 Lab 训练链路可用
Isaac Lab 楼梯展示配置	64/64 通过	用于最终视频展示；严格随机条件通过率较低

结果说明：Isaac Lab 虽然是更新的框架，但更换求解器、驱动器和导入链路后仍需要重新辨识和训练，不能假设“软件更新所以策略必然更好”。本项目保留了失败数字和边界条件，并通过兼容补丁、显式执行器、观测对齐和原生重训逐步缩小差距。

9. 大型上游仓库的工程化处理

Isaac Sim、Isaac Gym、Isaac Lab、RSL-RL 和 [unitree_rl_gym](#)

不直接复制到本仓库。原因是二进制体积大、许可证不同、更新困难，而且会掩盖真正的自主修改。

本项目改为：

- 1 在 [upstream.json](#) 固定上游 URL、tag 或 commit；
- 2 使用 [scripts/install_isaac_gym_overlay.py](#) 和 [install_isaac_lab_compat.py](#) 检查目标文件与版本；

- 3 先 dry-run 补丁，能匹配才应用；已经应用时自动识别；
- 4 只复制 TinyMal 覆盖层和机器人资源；
- 5 checkpoint、日志、视频和评估输出统一放进被 .gitignore 排除的 artifacts/；
- 6 大模型和视频通过 GitHub Release 或独立制品存储发布。

因此，GitHub 上每一处上游修改都能在 .patch 文件中直接查看，老师无需在数 GB 源码中寻找差异。

10. 如何审阅和复现修改

查看 Isaac Gym 注册表修改：

```
[bash]
sed -n '1,220p' backends/isaac_gym/patches/0001-register-tinymal-tasks.patch
```

查看 RSL-RL PPO 修改：

```
[bash]
sed -n '1,260p' backends/isaac_gym/patches/0003-reference-policy-distillation.patch
```

查看 Isaac Lab 兼容修改：

```
[bash]
sed -n '1,260p' backends/isaac_lab/patches/0001-isaac-sim-5.1-offline-compat.patch
```

应用 Isaac Gym 覆盖层：

```
[bash]
python scripts/install_isaac_gym_overlay.py \
--unitree-root _deps/unitree_rl_gym \
--rsl-rl-root _deps/rsl_rl
```

进行 MuJoCo 一步冒烟训练：

```
[bash]
RSL_RL_ROOT=_deps/rsl_rl \
python backends/mujoco/train_mujoco.py \
--num_envs 4 --max_iters 1 --benchmark
```

11. 总结与收获

本项目最重要的收获不是“调用 PPO

让机器人动起来”，而是建立了一套可诊断、可迁移、可复现的控制实验方法：

- 先进框架不自动带来先进结果，物理接口与执行器语义必须先对齐；

- 训练 reward 不是最终标准，必须加入跨引擎、楼梯、推力和随机动力学验收；
- sim2sim 的核心是观测、动作、关节顺序、控制周期和坐标系一致，而不是只保证网络形状相同；
- 多阶段学习容易遗忘旧能力，任务选择式策略蒸馏可保护已有步态；
- 大型依赖应使用固定版本、覆盖层和补丁管理，才能清楚证明自主修改并保持仓库轻量；
- 失败结果同样重要。Isaac Lab 的迁移差距明确指出了下一步应继续优化的方向：系统辨识、更强随机化、历史观测、teacher-student、对称正则和更严格的全任务统一验收。

本报告仅展示最关键的代码修改。更详细的源码、逐行补丁、安装方法、训练/评估脚本及持续更新见 GitHub：<https://github.com/Functionhx/actuatex>。